

Assignment 2: Parallel Triangle Counting using SIMD and OpenMP

Course: Parallel and Distributed Computing

CLO3 Perform analytical modelling, dependence, and performance analysis of parallel algorithms and programs.(

Instructions

- For your assignment, rename all your code files according to the following format e.g. ROLL-NUM-SECTION-TASK#-VERSION.EXT (23i-0001-A-TASK1-V1.c) and so on. Each file that you submit must contain your name, student-id, and assignment# on top of the file in comments.
- Combine all your work in one folder. The folder must contain only code and pdf file (no binaries, no exe files etc.).
- Rename the folder as ROLL-NUM-SECTION (e.g. 23i-0001-A) and compress the folder as a zip file (e.g. 23i-0001-A.zip). Do not submit .rar file.
- All the submissions will be done on Google classroom within the deadline. Submissions other than Google classroom (e.g. email etc.) will not be accepted.
- The student is solely responsible to check the final zip files for issues like corrupt files, viruses in the file, mistakenly exe sent. If we cannot download the file from Google classroom due to any reason, it will lead to zero marks in the assignment.
- If there is a syntax error in code, zero marks will be awarded in that part of assignment.
- Displayed output should be well-formatted and well presented. Use appropriate comments and indentation in your source code.
- Understanding the assignment is part of the evaluation.
- Be prepared for viva or any other evaluation mechanism after the submission of assignment.
- *Late Submission & Plagiarism* policies will be applied as described in course outline.

Tip: For timely completion of the assignment, start as early as possible.

Background and Motivation

Triangle counting is a fundamental primitive in graph analytics and is widely used in network analysis, community detection, and graph mining. Modern shared-memory multiprocessor systems offer two complementary sources of parallelism: data-level parallelism through SIMD instructions and task/thread-level parallelism through OpenMP.

In this assignment, students will study, implement, and experimentally evaluate two state-of-the-art shared-memory triangle counting approaches drawn from the attached research papers. One paper focuses primarily on SIMD-accelerated intersection of adjacency lists, while the second paper emphasizes algorithmic and cache-aware optimizations together with OpenMP-based parallelization. Building upon these two works, students will design and implement a hybrid solution that combines both SIMD and OpenMP techniques.

The goal of this assignment is to develop a deep understanding of how low-level architectural features and classical shared-memory parallel programming techniques can be combined to obtain scalable performance on real graph workloads.

Objectives

Upon completion of this assignment, students will be able to:

- Implement a baseline triangle counting algorithm for undirected graphs.
- Implement a SIMD-accelerated intersection kernel for triangle counting.
- Implement an OpenMP-parallel triangle counting algorithm following a modern shared-memory design.
- Design and implement a hybrid triangle counting solution that integrates SIMD and OpenMP.
- Evaluate performance, scalability, and correctness on representative graph datasets.
- Critically analyze the impact of architectural and algorithmic optimizations.

Problem Description

You are given two research papers (attached with this assignment):

- A paper focusing on SIMD-based acceleration of adjacency list intersections for triangle counting on shared-memory systems.
- A paper exploring optimized list-based and hash-based triangle counting approaches and their OpenMP parallelization.

Architecture Awareness and SIMD Capability Discovery

Before implementing the triangle counting kernels, students must explicitly identify the SIMD and threading capabilities of their target machine.

In particular, each group must:

- determine whether the target processor supports AVX, AVX2, AVX-512 (on x86) or NEON (on ARM);
- identify the SIMD register width and the number of single-precision or double-precision lanes available per vector register;
- identify the number of physical cores and hardware threads available on the system;
- decide and justify the number of OpenMP threads used in all experiments.

The detected architectural information must be reported in the experimental section of the report and must be used to guide the design of the SIMD kernels and the OpenMP configuration.

Your task is to implement and evaluate the following three versions of triangle counting for simple, undirected graphs:

Part A: SIMD-based version

Implement a triangle counting algorithm in which the core operation of intersecting two sorted adjacency lists is accelerated using SIMD instructions (e.g., SSE/AVX intrinsics or compiler-supported vectorization). The implementation should follow the intersection strategy described in the first paper.

Part B: OpenMP-SIMD based version

Implement a shared-memory parallel triangle counting algorithm using OpenMP SIMD clause.

- `#pragma omp simd`

The OpenMP SIMD version must be functionally equivalent to the intrinsic-based version and must be evaluated separately.

Students should clearly indicate which loops are vectorized using:

- explicit intrinsics (AVX or NEON), and
- OpenMP SIMD directives.

Part C: OpenMP-Thread based version

Implement a shared-memory parallel triangle counting algorithm using OpenMP. The implementation should follow the vertex- or edge-based parallelization strategy and the optimized ordering of vertices described in the second paper.

Part D: Hybrid SIMD + OpenMP version

Design and implement a hybrid triangle counting algorithm in which:

- OpenMP is used to parallelize the outer loops of the algorithm, and
- SIMD is used to accelerate the inner intersection kernels.

The hybrid version must clearly reuse and integrate the ideas developed in Parts A and B.

Thread Configuration Requirement

All OpenMP-based implementations must explicitly control and report the number of threads used (for example via `OMP_NUM_THREADS` or `omp_set_num_threads`).

The selected number of threads must be justified based on the detected hardware configuration.

Input Format and Datasets

- Graphs are simple and undirected.
- Input graphs are provided as edge lists.
- Self-loops and duplicate edges must be removed during preprocessing.

You may use following publicly available real-world graph datasets or instructor-provided datasets.

- <https://snap.stanford.edu/data/email-Eu-core.html>
- <https://snap.stanford.edu/data/ego-Facebook.html>
- <https://snap.stanford.edu/data/com-LiveJournal.html>
- <https://snap.stanford.edu/data/com-Friendster.html>

Functional Requirements

- Your code must correctly report the number of triangles in the graph.
- You must apply a vertex ordering strategy (e.g., degree-based ordering) to avoid duplicate counting.
- All three versions (SIMD, OpenMP, Hybrid) must be implemented and executable from a single code base or build system.
- The program must report:
 - number of vertices,
 - number of edges,
 - total triangle count,
 - execution time.

All implementations must count each triangle exactly once.

Evaluation Requirement

In addition to the evaluation described earlier, students must perform a comparative study of the following variants on their own machine:

1. SIMD-only version using intrinsics.
2. SIMD-only version using OpenMP SIMD directives.
3. OpenMP-only parallel version without explicit SIMD intrinsics.
4. Hybrid SIMD (intrinsics) + OpenMP version.
5. Hybrid OpenMP SIMD + OpenMP threading version.

For each dataset, students must report:

- execution time,
- achieved speedup relative to a scalar sequential baseline, and
- the best-performing variant on their machine.

Students must clearly identify:

Which variant performs best on their system and explain why, based on the observed SIMD width, memory behavior, and available parallelism.

The grade will explicitly include an assessment of:

- correctness of SIMD capability detection and reporting,
- appropriate selection of thread counts,
- proper use of OpenMP SIMD directives, and
- quality of the comparative performance analysis across all variants.

All experiments must be run on the same machine and under the same compilation settings.

Deliverables

1. Source code

- Well-structured and well-documented C/C++ code.
- Build instructions (Makefile or equivalent).

2. Technical report (maximum 8 pages)

The report must include:

- a brief description of the baseline algorithm,
- explanation of the SIMD intersection method used,
- explanation of the OpenMP parallelization strategy,
- design of the hybrid algorithm,
- experimental setup (hardware, compiler, flags, datasets),
- performance results (tables and figures),
- discussion and analysis of results.

3. Reproducibility package

- scripts or instructions to reproduce the experiments and plots.

Assessment Rubric

Criterion	Marks
Hardware capability discovery,	5
Correct implementation of SIMD-based triangle counting	10
OpenMP SIMD usage and cross-variant comparison	10
Correct implementation of OpenMP-based triangle counting	10
Design and implementation of hybrid SIMD + OpenMP version	15
Design and implementation of hybrid OpenMP SIMD + OpenMP version	15
Correctness and robustness (handling duplicates, ordering, validation)	10
Experimental evaluation and scalability analysis	10
Quality of performance discussion and interpretation of results	10
Clarity, structure, and technical quality of the report	10
Reproducibility and build/run documentation	5
Total	110

Important Notes

- You must clearly indicate which parts of your code correspond to the SIMD kernel, the OpenMP parallelization, and the hybrid integration.
- Use static or dynamic OpenMP scheduling where appropriate and justify your choice.
- Any external libraries used must be clearly stated in the report.
- The focus of this assignment is on performance engineering and architectural understanding rather than purely algorithmic novelty.